Detecting a chain jam from a sensorless motor speed estimate

Doruk Yüksel, MSc student in AI Engineering at Jönköping University, BSc in Electrical and Electronics Engineering.
Technical note, October 2026.

When a saw chain jams, the motor drive should notice within a few milliseconds and stop pushing. I wanted to know how quickly a sensorless drive can see this in its own speed estimate without tripping on hard but normal cutting, and what the check costs on a Cortex-M4. Everything here is simulated. The motor is the 1.2 kW, 14 pole PMSM from a public KTH master thesis about a battery chainsaw motor [1]. The loads are synthetic because I could not find recorded load data, so the numbers compare methods and are not field results. I wrote a closed loop simulation in Python, two sensorless observers (the MRAS from the thesis and a back EMF observer with a PLL) and a detector on the mean deceleration over a short window, and ported the observers and the detector to float32 C.

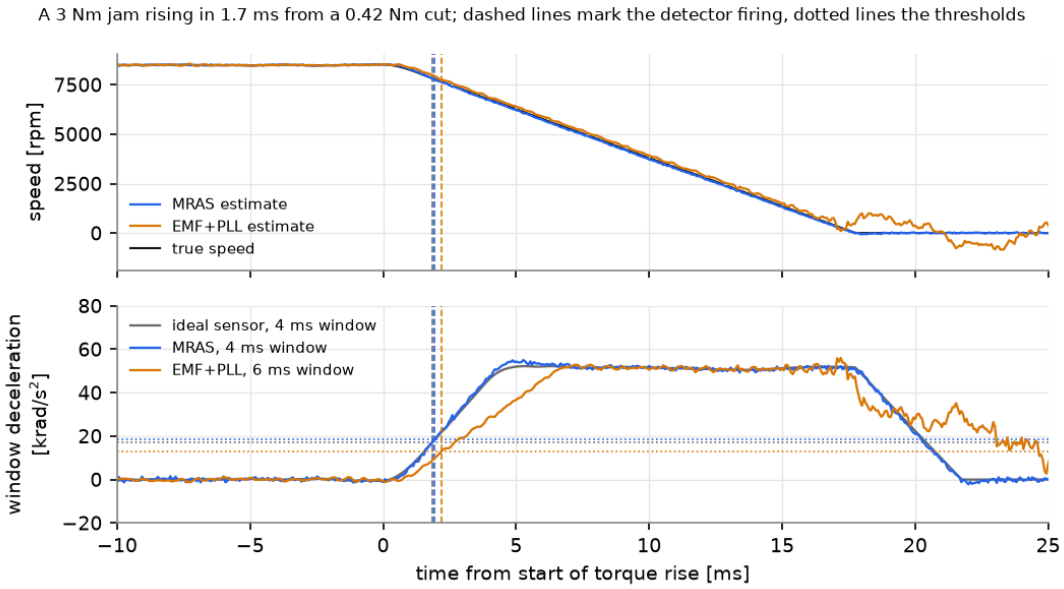


Figure 1. A jam that rises to 3 Nm in 1.7 ms from a 0.42 Nm cut, at 15 kHz. Top: the true rotor speed and the two sensorless estimates. Bottom: the windowed deceleration each detector sees, its threshold (dotted) and the moment it fires (dashed). All three fire 1.9 to 2.2 ms after the torque starts to rise. Near standstill the back EMF estimate wanders, so the detection has to happen well before that.

Main results

A plain threshold on the deceleration of a single sample does not work. A 1.4 Nm load step, which the motor can carry, slows the rotor at about 45 krad/s² during the first millisecond, until the speed loop catches up. A 2 Nm jam slows it at only about 17 krad/s², but it keeps doing so. A threshold high enough to ignore the load step misses many jams. At 15 kHz it caught 62% of 200 jams with the MRAS and 84% with the back EMF observer. Even an ideal speed sensor only reached 75%, and still had false alarms in 2 of 40 runs.

Taking the mean deceleration over a 4 to 6 ms window fixes this. At 15 kHz the windowed detector caught all 200 jams (2 to 6 Nm) with every speed source. The median latency from the start of the torque rise was 2.1 ms for the ideal sensor and the MRAS, and 2.5 ms for the back EMF observer. There were no false alarms in 160 s of normal cutting that was not used for tuning, and every snag of 2 Nm and above was caught. The results at 10 and 20 kHz are close. The windowed detector had one false alarm in the whole benchmark: the back EMF observer at 10 kHz, once in 160 s.

The sensorless estimate costs little latency. Across the three sampling rates the MRAS adds 0.05 to 0.1 ms to the median latency of an ideal sensor, and the back EMF observer 0.4 to 0.6 ms. Both observers know R, L and the flux exactly and start converged, so this is a best case.

On a Cortex-M4F the observer and the detector together take 340 to 364 instructions per control period, roughly 480 to 540 cycles. On a 100 MHz core running the loop at 15 kHz that is about 8% of the period. The library is 1.5 KB of code, uses no heap and no libm, and matches the Python reference within 1.0e-5 rad in angle on recorded closed loop runs.

The code, the results and the figures are at github.com/dorukyksl/sensorless-jam-detection. The script `run_all.sh` recreates the results, and GitHub Actions runs the tests, a short benchmark run and the instruction count on every push.

What is simulated

The motor data come from the thesis: R = 8.7 mΩ, L = 19 µH, flux linkage 2.37 mVs/rad, 7 pole pairs, rotor inertia 3.0e-5 kg m², rated 1.5 Nm at 8500 rpm from a 30 V link with 60 A maximum current. At rated speed the electrical frequency is 992 Hz, which leaves only 10 to 20 samples per electrical period. The drive is sampled FOC with the series PI current controllers of the thesis, one period of computation delay, an inverter limited to 30/√3 V, 0.2 A of current noise and a 12 bit ADC. The speed gains printed in the thesis appendix give 19% overshoot and let the speed go through zero under a 1 Nm load, so I tuned the speed loop myself to an 85 Hz crossover. That gives 464 rpm of droop under 1 Nm, close to the 450 rpm the thesis reports.

Normal cutting runs at 0.3 to 1.4 Nm with noise, knots and a ripple locked to the rotor angle. 30% of the cuts are hard entries that rise in 0.5 to 2 ms, and those are what make the problem hard. A jam rises in 0.5 to 3 ms to 2, 2.5, 3, 4 or 6 Nm and stays there, which is more than the motor can deliver. A snag is a half sine pulse of 1 to 6 Nm lasting 3 to 15 ms. The loads act like friction, so a large load stalls the rotor instead of turning it backwards.

Observers, detector and tuning

Both observers step the stator current model exactly over one period, with speed and voltage held. At rated speed the rotor turns 18 to 36 electrical degrees per sample, and a forward Euler step of the same model has a pole of magnitude 1.03 to 1.14, outside the unit circle. The MRAS follows the thesis (eq. 4.7 to 4.9), with its PI gains placed from the linearised loop. The second observer is a Luenberger observer that treats the back EMF as a constant disturbance, followed by a type 2 PLL. The detector fires when $(\omega[k-N] - \omega[k]) / (N \cdot T_s)$ is above a threshold while the estimated speed ω is above 20% of rated. With N = 1 it is the plain single sample threshold. A granted patent, US 12,103,201 B2 [2], uses the same kind of rule on a handheld power cutter to start braking.

Every combination of sampling rate, speed source, observer bandwidth (200, 400 and 800 Hz) and window (one sample up to 8 ms) is tuned the same way. The threshold is 1.1 times the largest windowed deceleration in 40 runs of 4 s of normal cutting. The bandwidth and window are then picked on a small separate set of events. False alarms are counted on another 40 runs of normal cutting and detections on 400 new events, and neither set is used for tuning. A jam only counts if the detector fires before the true speed drops below 20% of rated, because a back EMF estimate cannot be trusted below that. With windows of 0.5 to 2 ms the threshold has to sit above the transient after a hard normal load step, and 28 to 37 of the 40 jams at 2 Nm are missed. From 4 ms on every jam is caught, for a fraction of a millisecond more latency.

Table 1. Results at 15 kHz on the sets not used for tuning: 200 jams of 2 to 6 Nm, 160 snags of 2 to 6 Nm and 160 s of normal cutting. The last column is the single sample threshold.

Speed source	Window	Threshold [krad/s²]	False alarms	Jams	Snags	Latency, median and p95 [ms]	Single sample
Ideal sensor	4 ms	17.0	0	200/200	160/160	2.1, 3.9	150/200, false alarms in 2 runs
MRAS, 400 Hz	4 ms	18.6	0	200/200	160/160	2.1, 4.2	124/200
Back EMF + PLL, 800 Hz	6 ms	13.0	0	200/200	160/160	2.5, 4.7	167/200

The instruction counts come from arm-none-eabi-gcc 16.2 with -O2: 306 per call for the MRAS, 330 for the back EMF observer and 34 for the detector. The cycles weight each instruction with the Cortex-M4 timings at zero wait states, so they are an estimate. In one braking example outside the benchmark, the controller switched to 60 A on the q axis after the MRAS detector fired on a 3 Nm snag, and the rotor stopped within 0.6 revolutions. Without detection it turned 16.5 revolutions in the same 124 ms.

Limits and next steps

The threshold comes from 160 s of tuning data with a 10% margin, and that is not a guarantee. One of the nine tuned configurations did trip once on held out data, and a real product would need hours of recorded cutting. Only the rotor inertia is known, and the chain and sprocket would lower every deceleration and move every threshold. Dead time, PWM ripple, battery sag, stator resistance rising with temperature, startup and trigger release are not modelled. Next I want to add an extended Kalman filter as a third observer with the same C port and instruction count, since the thesis left it out because of its expected computational load, and rerun everything with stator resistance +50%, two and four times the inertia, and dead time.

References

[1] I. Westin. Sensorless Control of a PMSM: Evaluation of Different Speed and Position Estimation Methods. MSc thesis, KTH Royal Institute of Technology, 2016. diva2:1032918.

[2] US 12,103,201 B2, granted 2024. Deceleration threshold on a rotor speed estimated from the motor currents, followed by electromagnetic braking.